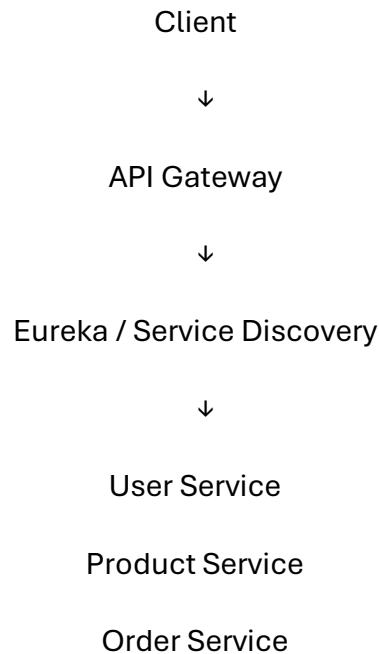


Task 4 — API Gateway

Now we will create the **API Gateway** for your microservices project.

Your architecture will be:



We will implement **routing, service forwarding, request filtering, authentication preparation, and error handling**.

Step 1 — Create API Gateway project

Open **Spring Initializr**.

Create a new project:

Name: api-gateway

Artifact: api-gateway

Language: Java

Java: 17

Packaging: Jar

Add these dependencies:

Spring Cloud Gateway

Eureka Discovery Client

Then:

Generate → Extract ZIP → Import into Eclipse

Step 2 — Check pom.xml

Your pom.xml should contain the Gateway and Eureka Client dependencies.

For example:

```
<dependency>

  <groupId>org.springframework.cloud</groupId>

  <artifactId>spring-cloud-starter-gateway</artifactId>

</dependency>
```

```
<dependency>

  <groupId>org.springframework.cloud</groupId>

  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>

</dependency>
```

Step 3 — Configure application.yml

Go to:

api-gateway
→ src/main/resources
→ application.yml

Put:

```
spring: application: name: api-gateway
```

```
cloud: gateway: routes:
```

- id: user-service
uri: lb://user-service
predicates:
 - Path=/api/users/**

- id: product-service
uri: lb://product-service
predicates:
 - Path=/api/products/**

- id: order-service
uri: lb://order-service
predicates:
 - Path=/api/orders/**

```
server: port: 8080
```

```
eureka: client: service-url: defaultZone: http://localhost:8761/eureka/
```

Step 4 — Understand the routing

For example, the client sends:

<http://localhost:8080/api/users/1>

The Gateway receives it and finds:

user-service
through Eureka

Then:

Client

↓

API Gateway :8080

↓

Eureka

↓

User Service :8081

Similarly,

/api/products/**

product-service

And

/api/orders/**

order-service

The lb:// means the Gateway uses **service discovery/load balancing** rather than a fixed address.

Step 5 — Request Filtering

For a simple request filter, create:

src/main/java

→ com.example.apigateway

→ filter

→ RequestLoggingFilter.java

Use

```
package com.example.apigateway.filter;
```

```
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
```

```
import org.springframework.web.server.ServerWebExchange;
```

```
import org.springframework.cloud.gateway.filter.GlobalFilter;
```

```
import reactor.core.publisher.Mono;
```

```
public class RequestLoggingFilter implements GlobalFilter {
```

```
    @Override
```

```
    public Mono<Void> filter(
```

```
        ServerWebExchange exchange,
```

```
        GatewayFilterChain chain) {
```

```
        System.out.println(
```

```
            "Request: " + exchange.getRequest().getURI()
```

```
        );
```

```
        return chain.filter(exchange);
```

```
    }
```

```
}
```

Then create:

GatewayConfig.java

```
package com.example.apigateway;
```

```
import com.example.apigateway.filter.RequestLoggingFilter;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
public class GatewayConfig {
```

```
    @Bean
```

```
    public RequestLoggingFilter requestLoggingFilter() {
```

```
        return new RequestLoggingFilter();
```

```
    }
```

```
}
```

Now the Gateway can log incoming requests.

Step 6 — Authentication Preparation

Here, we can prepare the Gateway to receive an authentication header.

The client can send:

Authorization: Bearer <token>

For now, we don't need to implement a complete JWT authentication system.

The Gateway can check whether the header exists.

Create:

AuthenticationFilter.java

```
package com.example.apigateway.filter;

import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.web.server.ServerWebExchange;
import org.springframework.http.HttpStatus;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import reactor.core.publisher.Mono;

public class AuthenticationFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(
        ServerWebExchange exchange,
        GatewayFilterChain chain) {

        String path = exchange.getRequest().getURI().getPath();

        if (path.startsWith("/api/")) {

            String authorization =
                exchange.getRequest()
                    .getHeaders()
                    .getFirst("Authorization");

            if (authorization == null ||
                !authorization.startsWith("Bearer ")) {

                exchange.getResponse()
                    .setStatusCode(HttpStatus.UNAUTHORIZED);

                return exchange.getResponse().setComplete();
            }
        }
    }
}
```

```
        return chain.filter(exchange);
    }
}
```

Then register it in GatewayConfig:

```
@Bean
public AuthenticationFilter authenticationFilter() {
    return new AuthenticationFilter();
}
```

This is **authentication preparation**. Later, you can replace this basic check with proper JWT/OAuth2 validation.

Step 7 — Error Handling

Create:

GatewayErrorHandler.java

A simple approach is to configure Gateway fallback/error routes later. For this task, first make sure invalid routes return an appropriate HTTP error instead of reaching a service that doesn't exist.

You can test:

<http://localhost:8080/api/unknown>

and verify that the Gateway returns an error response.

Step 8 — Start everything

Start the applications in this order

1. Service Registry :8761
2. User Service :8081
3. Product Service :8082
4. Order Service :8083
5. API Gateway :8080

Open,

[http://localhost:8761\](http://localhost:8761/)

You should see:

API-GATEWAY

USER-SERVICE

PRODUCT-SERVICE

ORDER-SERVICE

Step 9 — Test through the Gateway

use the Gateway:

<http://localhost:8080/api/users/1>

<http://localhost:8080/api/products/1>

<http://localhost:8080/api/orders/1>

The flow is:

